# Rockchip RK2118 杜比开发指南

文件标识：RK-JC-YF-F02

发布版本：V1.0.0

日期：2024-09-19

文件密级：□绝密 □秘密 □内部资料 ■公开

**免责声明**

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

**商标声明**

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

**版权所有 © 2024 瑞芯微电子股份有限公司**

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

## 前言

### 概述

本文主要介绍了如何在 RK2118 平台上完成安全启动的配置流程，涵盖密钥生成、固件签名以及安全启动机制的相关操作步骤，旨在帮助开发者顺利实现杜比功能的配置。

### 产品版本

| 芯片名称 | 内核版本 |
| --- | --- |
| RK2118 | RT-Thread v4.1.x |

### 读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

### 修订记录

| 版本号 | 作者 | 修改日期 | 修改说明 |
| --- | --- | --- | --- |
| V1.0.0 | tzb,hh | 2024-09-19 | 初始版本 |

目录

**Rockchip RK2118 杜比开发指南**

1. 杜比芯片库申请流程
    - 1.1 生成杜比传递公钥/私钥对
    - 1.2 导出杜比传递公钥
    - 1.3 解密传递公钥加密的杜比库
2. 使能安全启动
    - 2.1 放置解密后的杜比库
    - 2.2 生成安全启动签名密钥对
        - 2.2.1 生成 RSA 2048 公私钥对
    - 2.3 签名 `update.img`
        - 2.3.1 选择芯片类型
        - 2.3.2 选择 RSA 公私钥
        - 2.3.3 签名 Flag 配置
        - 2.3.4 update.img 固件签名
        - 2.3.5 使用 `sign.py` 简化签名流程
        - 2.3.6 确认安全启动
        - 2.3.7 确认 DSP2 杜比库
3. 附录
    - 3.1 安全启动过程

# 1. 杜比芯片库申请流程

RK2118支持Dolby Atoms功能的芯片型号为**RK2118G-Y**。对应的 DSP 杜比芯片库需要由客户通过瑞芯微业务渠道申请。客户需向瑞芯微业务提供相关信息，瑞芯微业务将提交这些材料到瑞芯微的内部 OA 系统进行审核。

以下列出了客户需提供的部分资料和信息，具体清单请联系瑞芯微业务以获取完整要求：

| 资料和信息 | 说明 |
| --- | --- |
| 申请出货的杜比芯片型号 | 申请的具体杜比芯片型号（如 RK2118G-Y） |
| 申请支持的杜比技术 | 根据杜比 SQA 表格填写完整的技术名称 |
| 合格终端客户(中文全称) | 终端客户的中文全称 |
| 合格终端客户(英文全称) | 终端客户的英文全称 |
| 合格终端客户中英文名称一致的证明 | 证明终端客户中英文名称一致的文件 |
| 终端产品名称 | 终端产品的名称 |
| 终端产品型号 | 终端产品的具体型号 |
| 合格终端客户营业执照/注册证书 | 客户的营业执照或注册证书 |
| 合格终端客户杜比资质证明文件 | 证明客户具备杜比资质的相关文件 |
| 合格终端客户指定接收库的邮箱 | 终端客户用于接收库的指定邮箱 |

审核通过后，瑞芯微将通过 OA 系统中填写的客户指定邮箱发送邮件通知，客户需提供杜比库传递用的公钥（RSA 2048）给瑞芯微。

客户回复邮件并提供杜比库传递的公钥（RSA 2048）后，瑞芯微将使用该公钥对杜比库进行加密，并将加密后的库上传至 FTP 的指定目录。瑞芯微会提供单独的账号和密码，供客户下载加密后的库。

客户下载并获取到使用公钥加密的杜比库后，需使用其对应的私钥解密。解密后的杜比库应放置在 SDK 指定的目录中。需要注意的是，杜比功能的启用必须依赖于安全启动，若未使能安全启动，杜比功能将无法正常使用。

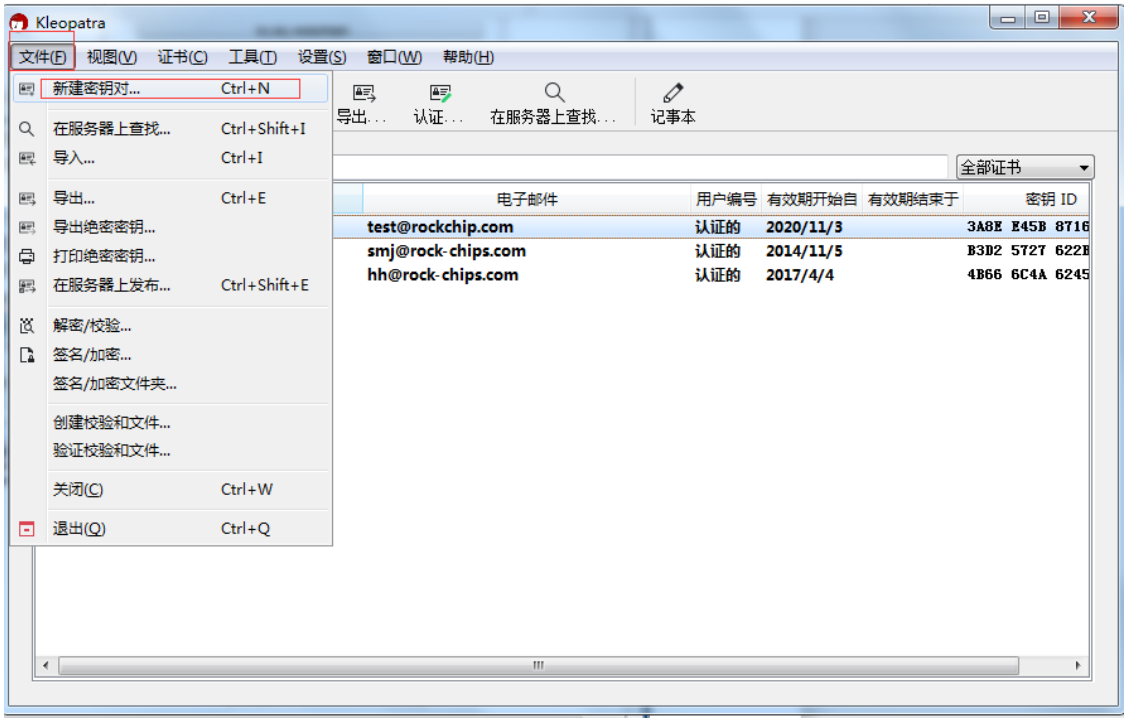
## 1.1 生成杜比传递公钥/私钥对

Kleopatra 是一款开源的加密与签名工具，提供了直观的图形界面，帮助用户轻松创建、导入、导出并管理加密密钥对。

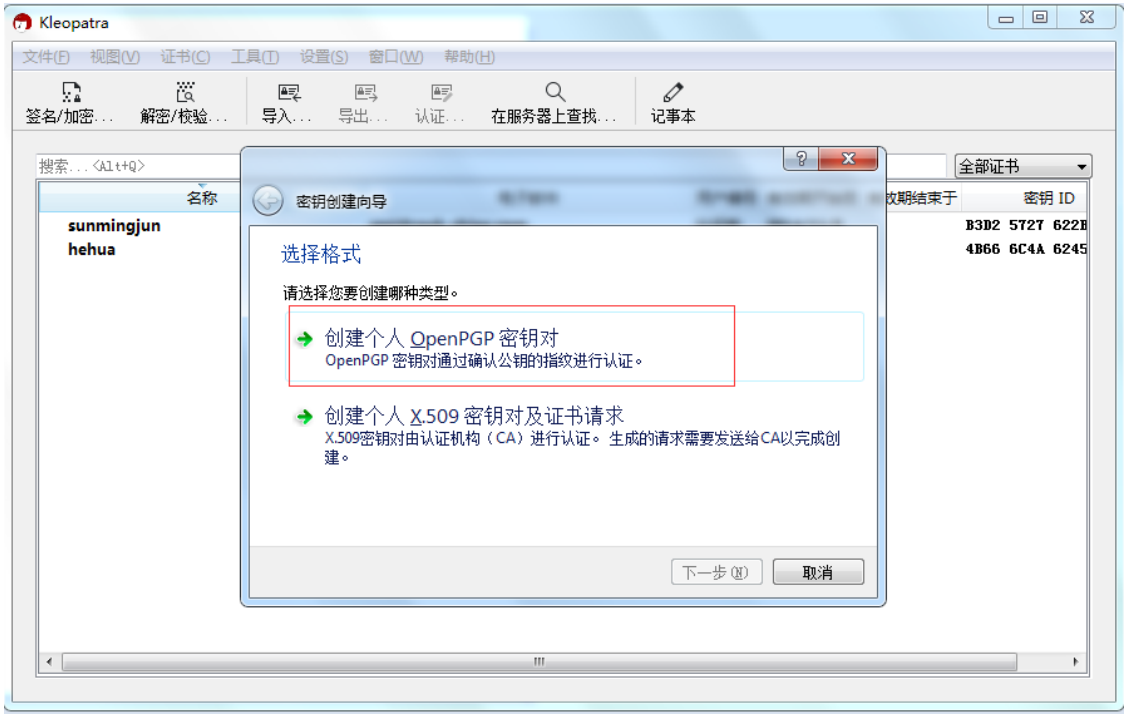
Kleopatra 是 Gpg4win 软件包的一部分，您可以从 Gpg4win 官网 下载最新版本。

安装并启动 Kleopatra 后，请按照以下步骤生成公私钥对：

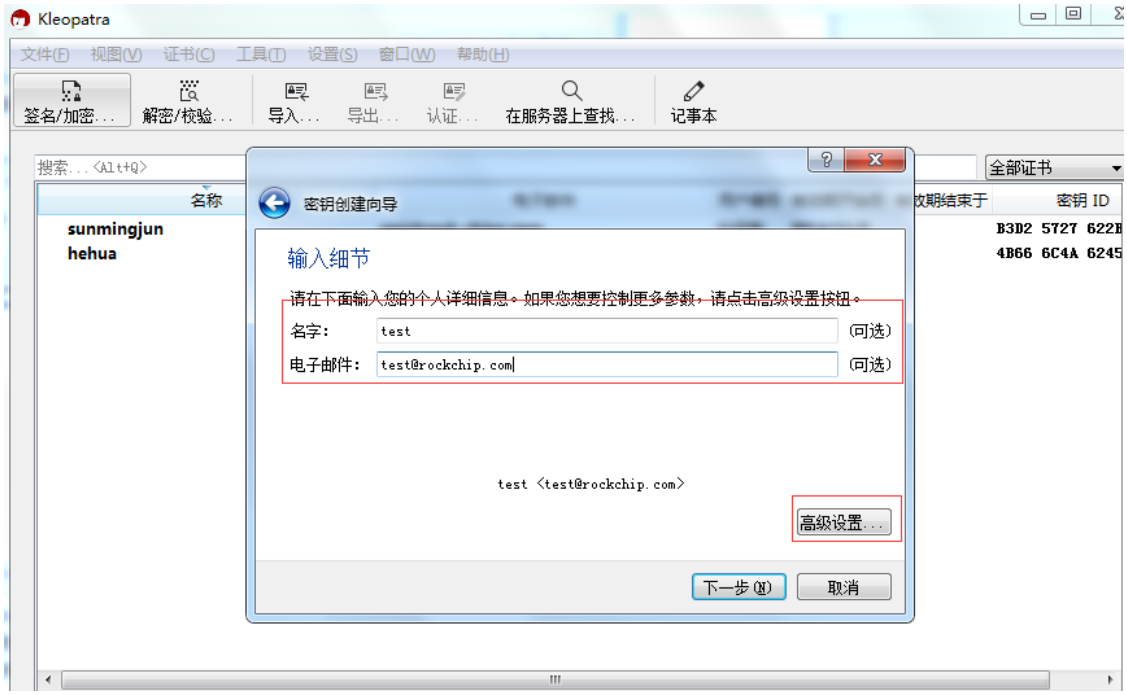
- 在 Kleopatra 工具中，点击“文件”。
- 选择“新建密钥对”以生成一对 RSA 加密密钥。



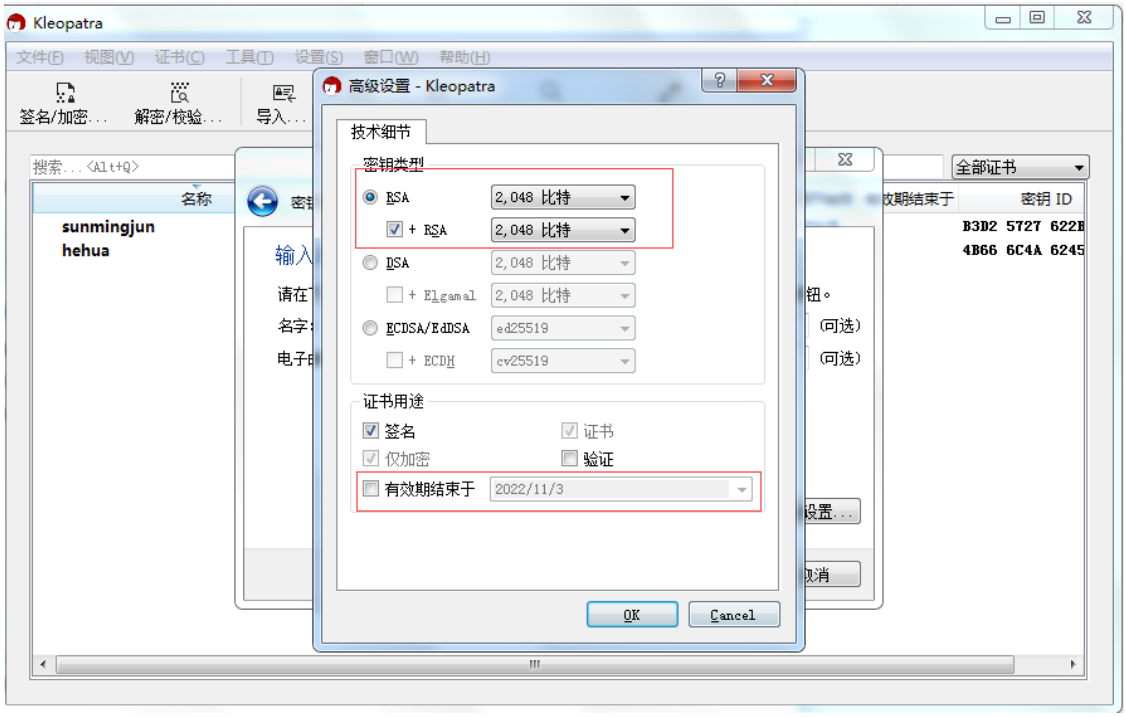

- 在弹出的密钥创建向导中，选择“创建个人 OpenPGP 密钥对”：




- 接着，填写姓名和邮件地址：




- 然后，点击“高级设置”：

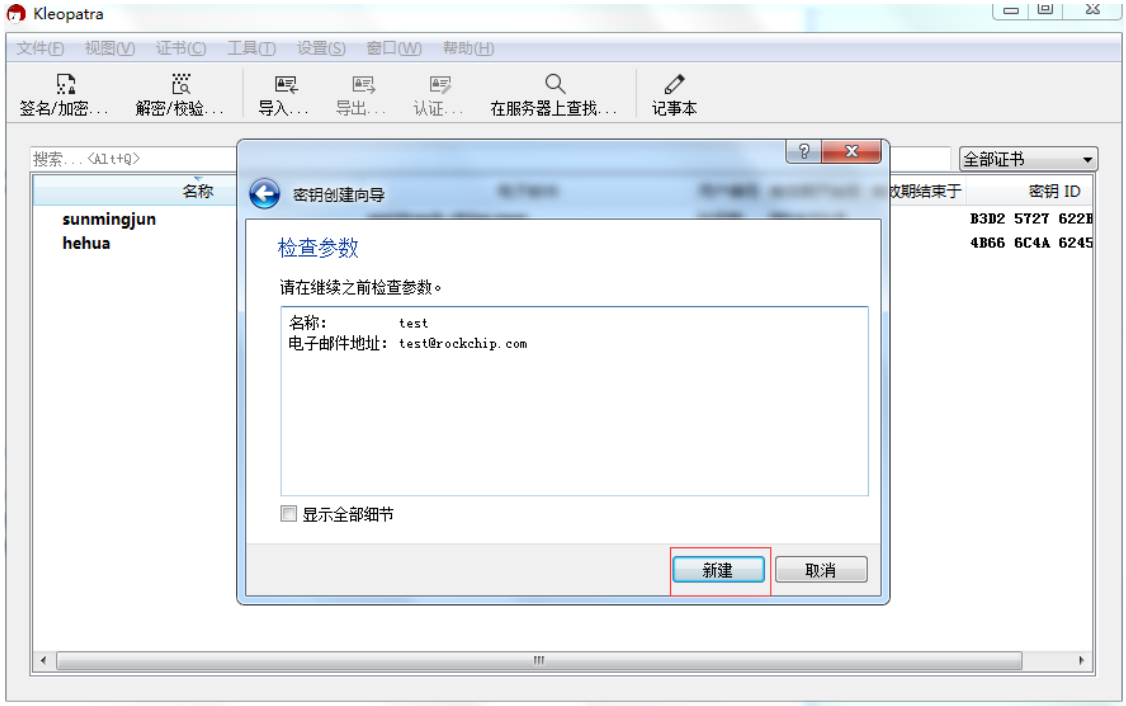



- 在高级设置中：

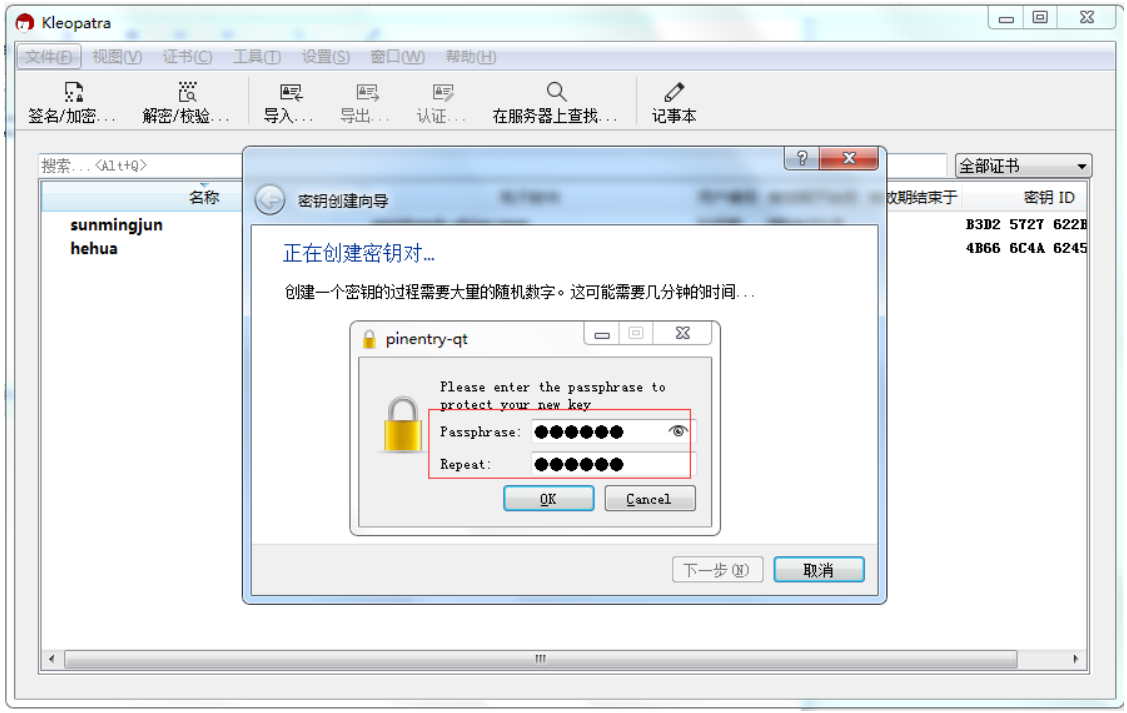
  选择 **RSA 算法**，并设置为 **2048** 位。

  取消勾选“有效期结束于 xxx”，避免密钥对自动失效。如需限制有效期，可根据需要设置日期。

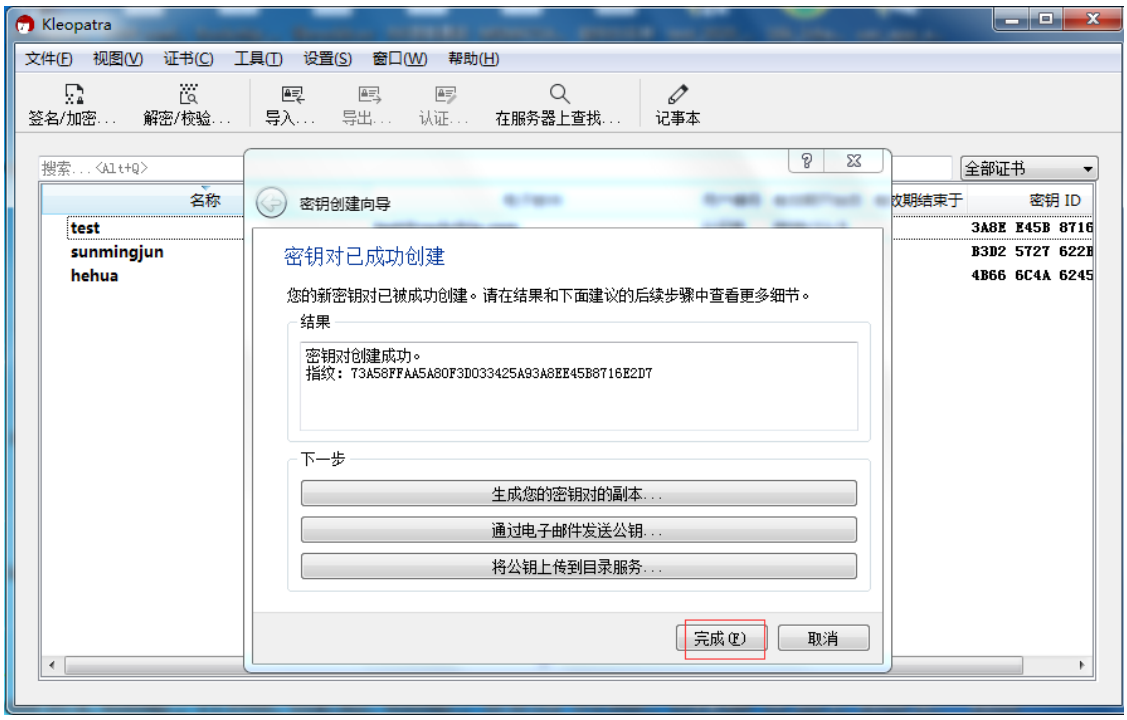
  点击“新建”以创建密钥对：




- 在弹出的对话框中输入密码（请注意，该密码非常重要，客户需妥善保存，后续解密时将需要使用此密码）：




- 点击“OK”后，Kleopatra 会生成密钥对：




生成密钥后，您可以按需进行以下操作：

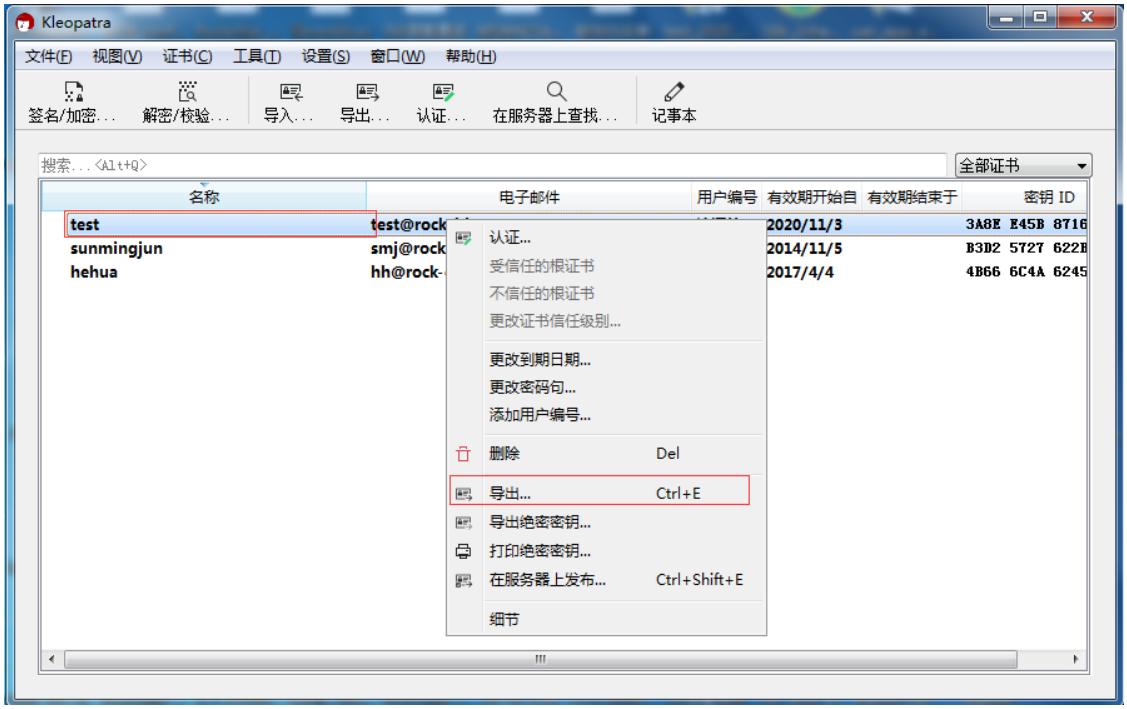
- **备份密钥**：将密钥保存到安全的存储位置。
- **通过邮件发送公钥**：点击选项将公钥发送给指定收件人。
- **将公钥备份到本地目录**。

**注意**：在实际操作中，通过邮件发送公钥时，有时可能会遇到邮件中未附加公钥的情况。

## 1.2 导出杜比传递公钥

要导出杜比传递的公钥，请按照以下步骤操作：

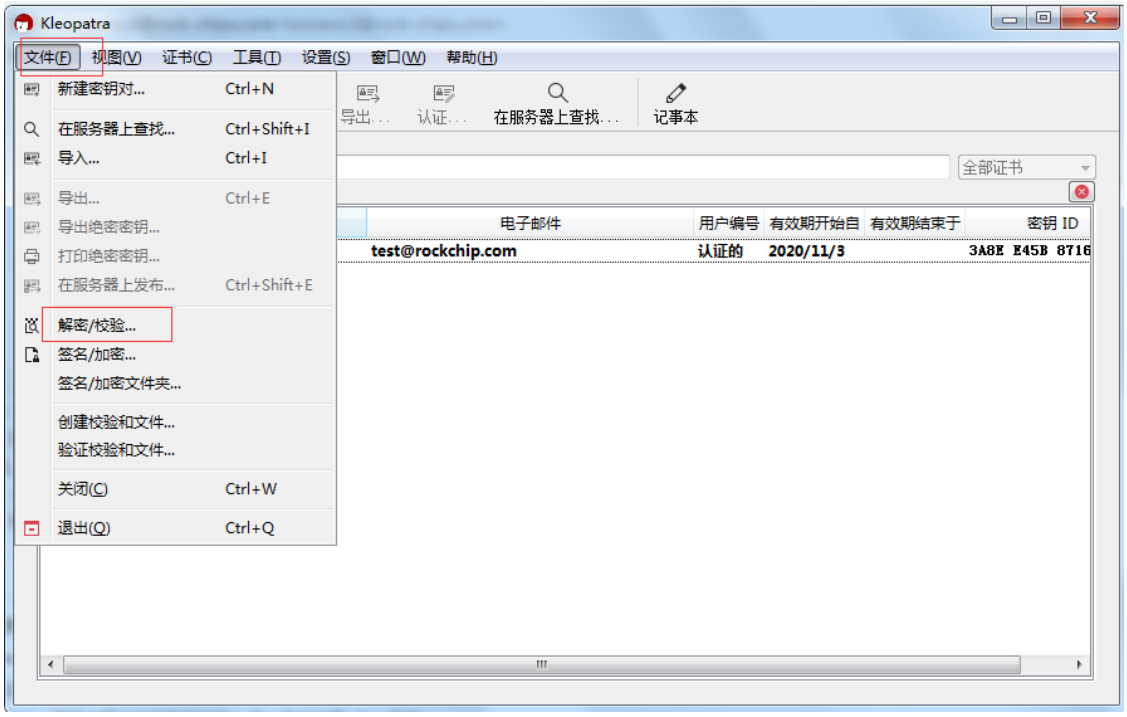
- 选择生成的密钥对，右键点击它。
- 在弹出的菜单中选择“导出”。
- 选择合适的目录，保存公钥文件（文件后缀为 `.asc`），例如：`test_0x8716E2D7_public.asc`。
- 将该公钥文件提供给瑞芯微。



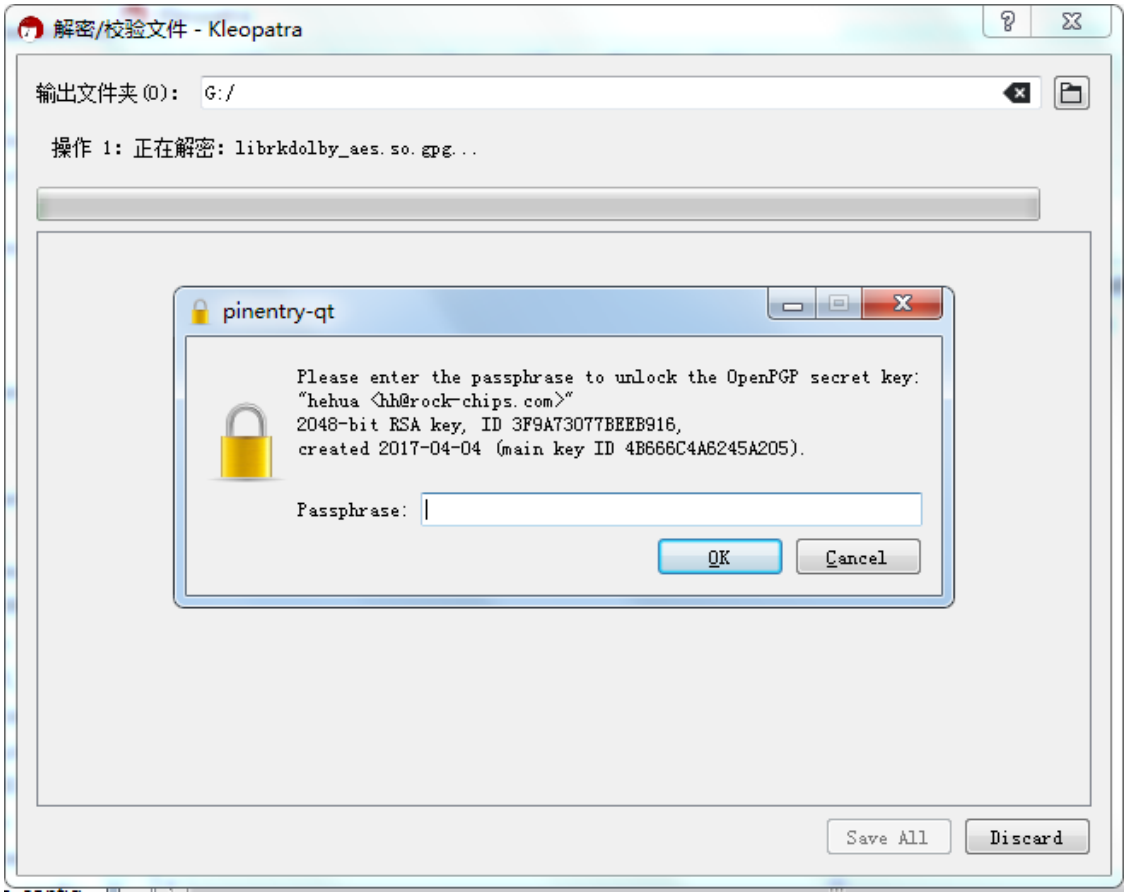

## 1.3 解密传递公钥加密的杜比库

根据杜比的要求，传递杜比库给客户时，需使用客户的传递公钥进行加密。客户通过 FTP 下载到加密的杜比库后，使用对应的私钥进行解密。具体步骤如下：

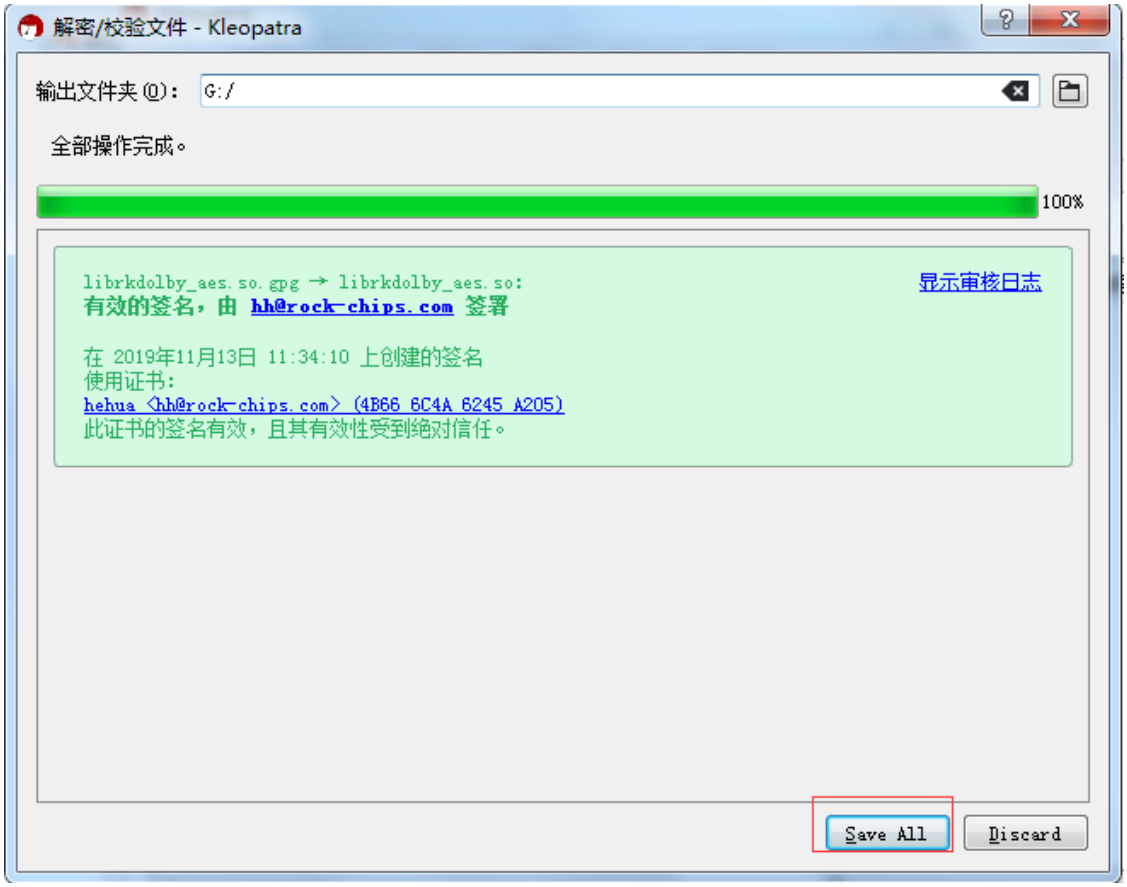
- 在 Kleopatra 中，选择“文件”菜单，点击“解密/校验”：




- 在弹出的对话框中，输入创建密钥时设置的密码：




- 解密成功后，选择要保存解密后文件的目录，点击“Save All”保存解密后的文件：




# 2. 使能安全启动

在 RK2118 芯片上，使用 **DSP2** 支持杜比技术，解密后的杜比库文件名为 `dsp2.bin`。

**注意**：启用杜比功能必须使能安全启动。如果未启用安全启动，杜比功能将无法正常使用。

## 2.1 放置解密后的杜比库

以 **EVB3_V10 板子** 为例，其打包配置文件路径为：

```
bsp/rockchip/rk2118/board/evb3_v10/package-file
```

该文件定义了打包时各组件的相对路径。在解密杜比库后，需将解密后的 `dsp2.bin` 文件放置在 `package-file` 文件中指定的路径下。

例如，`package-file` 中指定了 `dsp2_a` 和 `dsp2_b` 的存放路径为：

```
dsp2_a
../../../../../components/hifi4/rtt/dsp_fw/evb_soundbar_demo/dsp2.bin
dsp2_b
../../../../../components/hifi4/rtt/dsp_fw/evb_soundbar_demo/dsp2.bin
```

因此，解密后的 `dsp2.bin` 文件应放置在以下路径：

```
../../../../../components/hifi4/rtt/dsp_fw/evb_soundbar_demo/dsp2.bin
```

按照 `package-file` 中指定的路径，将解密后的文件放入上述目录中。

## 2.2 生成安全启动签名密钥对

签名工具路径为：

```
bsp/rockchip/tools/sign_tool
```

### 2.2.1 生成 RSA 2048 公私钥对

在工具目录下执行以下命令，生成 RSA 2048 密钥对：

```
$ ./sign_tool kk --bits=2048
********sign_tool ver 1.45********
bits is 2048
start to generate key...
saving private key at
/home/tzb/work/rk/sdk/rk2118/dev/bsp/rockchip/tools/sign_tool/private_key.pem...
saving public key at
/home/tzb/work/rk/sdk/rk2118/dev/bsp/rockchip/tools/sign_tool/public_key.pem...
generating key ok.
```

生成的 `private_key.pem` 为私钥，`public_key.pem` 为公钥，请妥善保存签名密钥。

## 2.3 签名 `update.img`

### 2.3.1 选择芯片类型

签名前需要选定芯片类型。选定后会记录配置，若后续签名操作的芯片类型未发生变化，则无需再次选择。

```
$ ./sign_tool cc --chip=2118
********sign_tool ver 1.45********
set chip is 2118
setting chip ok.
```

### 2.3.2 选择 RSA 公私钥

签名前需要选择 RSA 密钥对。选定后会记录配置，若后续签名操作的密钥对未发生变化，则无需再次选择。

```
$ ./sign_tool lk --key private_key.pem --pubkey public_key.pem
********sign_tool ver 1.45********
private key is private_key.pem
public key is public_key.pem
loading key ok.
```

### 2.3.3 签名 Flag 配置

为了启用安全启动，必须配置签名 Flag。在此示例中，开启 `loader` 的写 OTP 功能。固件在首次烧录启动时，会写入 OTP 以使能安全启动，并写入签名公钥的哈希值。

```
$ ./sign_tool ss --flag=0x20
********sign_tool ver 1.45********
sign flag is 0x20
setting sign argument ok.
```

### 2.3.4 update.img 固件签名

接下来对 `update.img` 固件包中的 `loader`、`tfm`、`os`、`dsp` 等内容进行签名。

```
$ cp ../../rk2118/Image/update.img .
$ ./sign_tool sf --firmware update.img
********sign_tool ver 1.45********
Firmware is update.img
parent process, pid = 261555
child process, pid = 261556
...
signing firmware ok
```

当前目录下的`updata.img`为签名后的固件。

### 2.3.5 使用 `sign.py` 简化签名流程

为了简化上述签名步骤，我们提供了一个名为 `sign.py` 的脚本。该脚本位于 `bsp/rockchip/rk2118/sign.py`，可以自动执行固件签名的所有必要步骤，包括：

- 选择芯片类型（`RK2118`）
- 选择RSA 公钥和私钥
- 签名Flag配置（值为 `0x20`，用于开启安全启动）
- updata.img 固件签名

通过使用 `sign.py`，用户无需手动输入多个命令即可完成固件的签名流程。

- 使用默认参数执行固件签名：

```
./bsp/rockchip/rk2118/sign.py
```

该命令会使用默认的 RSA 私钥 (`../tools/sign_tool/private_key.pem`)、公钥 (`../tools/sign_tool/public_key.pem`)，以及固件路径 (`./Image/update.img`)。

- 自定义密钥和固件路径进行签名：

```
./bsp/rockchip/rk2118/sign.py --key my_private_key.pem --pubkey
my_public_key.pem --firmware ./path/to/update.img
```

用户可以根据需求指定私钥、公钥和固件路径。

### 2.3.6 确认安全启动

在烧录签名后的 `update.img` 并启动设备后，可以通过启动日志确认安全启动的状态。日志示例如下：

**注意**：在启用了安全启动的设备上使用烧录工具`Rockchip Soc ToolKit`烧写`update.img`，如果签名验证失败，在烧录阶段就会出错停止。

```
Boot1 Release Time: Sep 14 2024 17:35:42, version: 1.25
ROM VER:2.0
dev_id:0x3 init.
SPI FLASH ID:FF B 31
sfc_nand id: B 31 0
sfc_nand A0 = 0x0
sfc_nand B0 = 0x13
read_lines = 2
prog_lines = 2
page_read_cmd = 6B
page_prog_cmd = 32
sfc_nand_density 40000
spi nand capacity: 128MB
GPT part:  0, name: loader, start:0x0, size:0x600
GPT part:  1, name: vnvm, start:0x600, size:0x200
GPT part:  2, name: cpu0_secure_a, start:0x800, size:0x200
GPT part:  3, name: cpu1_secure_a, start:0xA00, size:0x200
GPT part:  4, name: cpu0_secure_b, start:0xC00, size:0x200
GPT part:  5, name: cpu1_secure_b, start:0xE00, size:0x200
GPT part:  6, name: recovery, start:0x1000, size:0x400
```

```
GPT part:  7, name: misc, start:0x1400, size:0x400
GPT part:  8, name: cpu0_os_a, start:0x1800, size:0x800
GPT part:  9, name: cpu1_os_a, start:0x2000, size:0x400
GPT part: 10, name: cpu0_os_b, start:0x2400, size:0x800
GPT part: 11, name: cpu1_os_b, start:0x2C00, size:0x400
GPT part: 12, name: dsp0_a, start:0x3000, size:0x2800
GPT part: 13, name: dsp0_b, start:0x5800, size:0x2800
GPT part: 14, name: dsp1_a, start:0x8000, size:0x2800
GPT part: 15, name: dsp1_b, start:0xA800, size:0x2800
GPT part: 16, name: dsp2_a, start:0xD000, size:0x2800
GPT part: 17, name: dsp2_b, start:0xF800, size:0x2800
GPT part: 18, name: rkstudio_a, start:0x12000, size:0x200
GPT part: 19, name: rkstudio_b, start:0x12200, size:0x200
GPT part: 20, name: root_a, start:0x12400, size:0x1000
GPT part: 21, name: root_b, start:0x13400, size:0x1000
GPT part: 22, name: userdata, start:0x14400, size:0x2B7DF
SecureMode:1
Head flags=0x11
Secure enable:1, ret:0
find part:misc OK. first_lba:0x1400.
find part:misc OK. first_lba:0x1400.
slot priority:15, tries_remaining:0, successful_boot:1
slot priority:14, tries_remaining:3, successful_boot:0
MemBoot slot:0
load:cpu1_secure_a.
find part:cpu1_secure_a OK. first_lba:0xA00.
fw size:  12288
fw entry: 0x63F00000
Verify sign...
check image ret:0!
load data:
63F00000:  0022FFF8 63F02555 00000000 00000000
load:cpu1_os_a.
find part:cpu1_os_a OK. first_lba:0x2000.
fw size:  105984
fw entry: 0x63F08000
Verify sign...
check image ret:0!
load data:
63F08000:  A4000000 A3F087C1 A3F08819 A3F089B3
load:cpu0_os_a.
find part:cpu0_os_a OK. first_lba:0x1800.
fw size:  401408
fw entry: 0x60100000
Verify sign...
check image ret:0!
load data:
60100000:  A0F00000 A01007C1 A0100819 A01009B3
load:cpu0_secure_a.
find part:cpu0_secure_a OK. first_lba:0x800.
fw size:  138240
fw entry: 0x60000000
Verify sign...
check image ret:0!
load data:
60000000:  60080BF8 60002BD1 60002A65 60008631
start system...
```

- `SecureMode:1` 表示芯片已启用安全启动。
- `check image ret:0!` 表示签名验证通过，若验证失败，系统将停止启动，可以看到对cpu0/1的os和secure进行了签名验证。

### 2.3.7 确认 DSP2 杜比库

当 `cpu0_os` 成功启动后，系统会继续加载 `dsp2` 库。如果 `dsp2` 成功启动，日志中会打印出相关信息。示例日志如下：

```
[DSP2 INFO] I [pb_ai_adec_init: 844] build Sep 10 2024 10:57:00, is_soundbar(1)
atoms(1)
```

- `atoms(1)` 表示 `dsp2` 已成功启用杜比功能，表明当前是杜比授权的版本。

此日志可以帮助确认 `dsp2` 已经正确加载并启用了杜比功能。

# 3. 附录

## 3.1 安全启动过程

1. **验证公钥哈希值**

   ROM 代码从 loader 头部提取公钥，并计算其哈希值（SHA256）。然后将该哈希值与预先烧录在 OTP 存储中的公钥哈希值进行比较。如果两者相同，启动过程继续；如果不同，启动将被阻止。

2. **验证 loader 签名**

   ROM 代码使用步骤 1中验证的公钥（RSA2048）来验证 loader 的签名。如果签名验证成功，启动过程继续；否则，启动将被阻止。

3. **loader 验证公钥哈希值**

   loader 从自身头部再次提取公钥，计算其哈希值（SHA256），并与 OTP 中的公钥哈希值进行比较。如果两者相同，启动过程继续；否则，启动将被阻止。

4. **验证 RTOS 和 TF-M 签名**

   loader 使用步骤 3中验证的公钥来验证 RTOS 和 Trusted Firmware-M（TF-M）的签名。如果两个签名都验证成功，loader 将跳转执行 TF-M。TF-M 完成初始化后，会跳转到 RTOS，RTOS 接着调用 TF-M 来验证 DSP 固件。如果任何一步验证失败，启动将被阻止。